BDD SQL — Devoir 3

BUT3 GEA — IUT de Saint-Denis

Avant de commencer

Ce devoir porte sur les **fonctions d'agrégation** (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`), sur `GROUP BY` et sur `HAVING`. Il compte deux parties : la partie A utilise **chinook.db** ; la partie B utilise **votre base individuelle**, la même que pour le Devoir 2.

Les **notes de cours** (`notes-S1-S3.pdf`, jointes à ce devoir sur Teams) résument tout ce qu'il faut savoir : gardez-les ouvertes pendant que vous travaillez.

> **N'utilisez que ce qui a été vu en cours.** Vos requêtes doivent être écrites uniquement avec les éléments de SQL vus en séances 1 à 3.

Il y a plus d'exercices que ce que vous aurez probablement le temps de faire en séance — c'est normal. Faites-en le plus possible, dans l'ordre, et terminez chez vous si besoin.

Comment rendre votre travail :

1. Ouvrez un éditeur de texte simple (Bloc-notes, Notepad++, ou TextEdit sur Mac).
2. Pour chaque exercice, écrivez une ligne `-- exercice N` (où `N` est le numéro de l'exercice), suivie de votre réponse.
3. Si la réponse est une requête SQL, **testez-la d'abord dans SQLiteStudio** pour vérifier qu'elle fonctionne, puis copiez-collez-la telle quelle (sans `--` devant).
4. Si la réponse n'est **pas** une requête SQL (un nombre, un nom, une phrase), écrivez-la quand même précédée de `--`, comme un commentaire. Ainsi, on peut copier-coller tout votre fichier d'un coup dans l'éditeur SQL sans erreur : les lignes commentées sont ignorées.
5. Enregistrez le fichier sous le nom **Prénom-NOM-SQL-Devoir3.txt**.
6. Déposez le fichier sur Teams avant la date limite indiquée sur Teams.

Exemple de fichier attendu :

```
-- exercice 1
SELECT COUNT(*) FROM tracks;

-- exercice 6
-- 312

-- exercice 20
-- 9
-- Parce que ...
```

> **Vous n'avez plus votre base individuelle ?** Rendez-vous sur `https://github.com/frey-iutsd/sql-2026`, dossier `sql2`, puis `bdd`. Cliquez sur le fichier qui porte votre numéro étudiant, puis sur le bouton de téléchargement (« Download raw file »). Ouvrez-le dans SQLiteStudio avec **Ctrl+O**.

> **Deux bases ouvertes en même temps ?** Dans la barre d'outils de l'éditeur SQL, une liste déroulante indique sur quelle base la requête va s'exécuter. Vérifiez qu'elle affiche **chinook** pour la partie A, et **votre base** pour la partie B. Sinon, vous obtiendrez une erreur « no such table ».

Rappel express

```sql
-- une seule ligne de résultat, pour toute la table :
SELECT COUNT(*), SUM(Total), ROUND(AVG(Total), 2) FROM invoices;

-- une ligne de résultat par pays :
SELECT BillingCountry, COUNT(*) AS nb_factures
FROM invoices
GROUP BY BillingCountry;

-- en ne gardant que les pays ayant au moins 10 factures :
SELECT BillingCountry, COUNT(*) AS nb_factures
FROM invoices
GROUP BY BillingCountry
HAVING COUNT(*) >= 10;
```

`WHERE` filtre les **lignes** avant de faire les groupes ; `HAVING` filtre les **groupes** une fois qu'ils sont faits. L'ordre d'écriture est toujours : `SELECT` … `FROM` … `JOIN` … `WHERE` … `GROUP BY` … `HAVING` … `ORDER BY`.

Partie A — chinook.db

Exercice 1. Donnez une requête affichant le nombre total de pistes (table `tracks`).

Exercice 2. Donnez une requête affichant la durée moyenne d'une piste, **en minutes**, arrondie à 1 chiffre après la virgule. (La colonne `Milliseconds` donne la durée en millisecondes ; une minute = 60 000 millisecondes. Pensez au `.0` pour éviter une division entière.)

Exercice 3. Donnez une requête affichant, pour chaque genre musical, le **nom du genre** et le **nombre de pistes** de ce genre, triés du genre qui a le plus de pistes à celui qui en a le moins. (Le nom du genre est dans la table `genres` : il faut une jointure.)

Exercice 4. Reprenez l'exercice 3, mais n'affichez que les genres ayant **plus de 100 pistes**.

Partie B — votre base individuelle

Votre base contient trois tables : `clients` (`id`, `nom`, `ville`), `produits` (`id`, `nom`, `prix`) et `commandes` (`id`, `client_id`, `produit_id`, `montant`). Chaque commande concerne un client (`client_id`) et un produit (`produit_id`).

Chaque question « donnez une requête » est suivie d'une question qui vous demande de **rapporter le résultat** obtenu en exécutant votre requête. Comme votre base est différente de celle de vos camarades, vos réponses le seront aussi.

Compter et additionner

Exercice 5. Donnez une requête affichant le nombre total de commandes dans votre base.

Exercice 6. D'après le résultat de l'exercice 5 : combien y a-t-il de commandes ?

Exercice 7. Donnez une requête affichant le chiffre d'affaires total, c'est-à-dire la somme des montants de toutes les commandes, arrondie à 2 chiffres après la virgule.

Exercice 8. D'après le résultat de l'exercice 7 : quel est le chiffre d'affaires total ?

Exercice 9. Donnez **une seule** requête affichant, pour la table `produits`, le prix moyen (arrondi à 2 chiffres après la virgule), le prix le plus bas et le prix le plus haut, dans trois colonnes nommées `prix_moyen`, `prix_min` et `prix_max`.

Exercice 10. D'après le résultat de l'exercice 9 : quels sont ces trois prix ?

Regrouper avec GROUP BY

Exercice 11. Donnez une requête affichant, pour chaque ville, le nom de la ville et le nombre de clients qui y habitent (colonne nommée `nb_clients`), triés du plus grand nombre au plus petit.

Exercice 12. D'après le résultat de l'exercice 11 : combien de clients compte la ville qui en a le plus ?

Exercice 13. Reprenez l'exercice 11, mais n'affichez que les villes ayant **au moins 3 clients**.

Exercice 14. D'après le résultat de l'exercice 13 : combien de villes sont dans ce cas ?

Regrouper après une jointure

Exercice 15. Donnez une requête affichant, pour chaque produit, son nom, le nombre de commandes qui le concernent (colonne `nb_commandes`) et le chiffre d'affaires qu'il a rapporté, arrondi à 2 chiffres après la virgule (colonne `ca`), triés par chiffre d'affaires décroissant. (Jointure entre `commandes` et `produits`.)

Exercice 16. D'après le résultat de l'exercice 15 : quel produit a rapporté le plus, et combien ?

Exercice 17. Donnez une requête affichant, pour chaque client, son `id`, son nom, son nombre de commandes et le montant total de ses commandes, arrondi à 2 chiffres après la virgule (colonne `total`), triés par total décroissant. **Regroupez par `clients.id` et `clients.nom`** (pas seulement par le nom). (Jointure entre `clients` et `commandes`.)

Exercice 18. D'après le résultat de l'exercice 17 : quel est l'`id`, le nom et le montant total du client qui a le plus dépensé ?

Exercice 19. Donnez une requête affichant les noms de famille portés par **plusieurs** clients de votre base, avec, pour chacun, le nombre de clients qui portent ce nom.

Exercice 20. D'après le résultat de l'exercice 19 : combien de noms de famille sont dans ce cas ? Puis expliquez, en une ou deux phrases et avec vos propres mots, pourquoi l'exercice 17 demande de regrouper par `clients.id` et pas seulement par `clients.nom`.

Exercice 21. Donnez une requête affichant l'`id`, le nom et le nombre de commandes des clients ayant passé **au moins 5 commandes**.

Exercice 22. D'après le résultat de l'exercice 21 : combien de clients sont dans ce cas ?

Bonus

Exercice 23 (bonus). Donnez une requête affichant, pour chaque ville, le chiffre d'affaires total des commandes passées par les clients de cette ville (arrondi à 2 chiffres après la virgule), trié par chiffre d'affaires décroissant.

Exercice 24 (bonus). D'après le résultat de l'exercice 23 : quelle ville rapporte le plus, et combien ?

> **Exercice 25 (bonus).** En ne tenant compte que des commandes d'un montant **supérieur à 100**, donnez une requête affichant, pour chaque ville, le nombre de telles commandes passées par ses clients, en ne gardant que les villes où ce nombre vaut **au moins 5**. (Il faut à la fois un `WHERE` et un `HAVING`.)

> **Exercice 26 (bonus).** D'après le résultat de l'exercice 25 : combien de villes sont dans ce cas ?

N'oubliez pas de déposer votre fichier `Prénom-NOM-SQL-Devoir3.txt` sur Teams.